

# Manual Técnico — Filosofar

## 1. Stack y versiones

| Componente              | Versión                                                |
|-------------------------|--------------------------------------------------------|
| Kotlin                  | 2.0.20                                                 |
| Android Gradle Plugin   | 8.5.2                                                  |
| Compose compiler plugin | 2.0.20 (plugin<br>org.jetbrains.kotlin.plugin.compose) |
| Compose BOM             | 2024.09.00                                             |
| Material 3              | 1.2.1                                                  |
| Navigation Compose      | 2.7.7                                                  |
| Room                    | 2.6.1 (KSP)                                            |
| Coroutines              | 1.8.1                                                  |
| KSP                     | 2.0.20-1.0.25                                          |
| Gradle                  | 8.7 (wrapper)                                          |
| JDK                     | 17                                                     |
| compileSdk / targetSdk  | 34                                                     |
| minSdk                  | 24                                                     |

Todas las versiones son fijas (sin + ni latest) para builds reproducibles.

## 2. Arquitectura

MVVM + Repository, en tres paquetes principales bajo `com.educalab.filosofar`:

```
data/
  local/
    entity/      22 entidades Room (una o dos por archivo temático)
    dao/         Interfaces DAO con Flow para lecturas reactivas
    seed/        Contenido semilla en Kotlin (SeedIslands, SeedQuestions, ...)
  AppDatabase.kt
  repository/    Un repositorio por dominio funcional; traduce Entity <-> modelo de dominio
domain/
  model/         Data classes de dominio, sin anotaciones Room
  logic/         Motores puros: LogicChallengeEngine, ReasonCoherenceEngine,
                  ProgressCalculator, BadgeEngine – 0% Android, 100% testeable en JVM
ui/
  navigation/    Routes.kt + FilosofarNavHost.kt (Navigation Compose)
  theme/         Color.kt, Type.kt, Theme.kt, Shape.kt
  components/    Ilustraciones Compose Canvas reutilizables (sin Material Icons como base)
```

```

    screens/          Un paquete por módulo, cada uno con su ViewModel + Screen
util/
    ViewModelFactory.kt  Factory manual (sin Hilt)
    AudioRecorderManager.kt  MediaRecorder/MediaPlayer reales
AppContainer.kt  Contenedor de dependencias manual y explícito
FilosofarApp.kt  Application; instancia AppContainer en onCreate()
MainActivity.kt  Host único de Compose

```

**Regla de capas seguida:** los Composables nunca ejecutan SQL ni contienen reglas de negocio; los ViewModels exponen StateFlow y delegan en repositorios; los repositorios traducen entre entidades Room y modelos de dominio y orquestan los motores de domain/logic; los motores de dominio son funciones/objetos puros sin dependencias de Android, por lo que se prueban con JUnit puro sin Robolectric.

### 3. Inyección de dependencias

No se usa Hilt/Dagger. AppContainer (en la raíz del paquete) construye y expone, con by lazy, todos los repositorios a partir de los DAOs de AppDatabase.getInstance(context).

FilosofarApp.onCreate() crea una única instancia de AppContainer. Las pantallas obtienen sus ViewModels vía viewModel(factory = ViewModelFactory(container) { ... }), definido en util/ViewModelFactory.kt.

### 4. Base de datos (Room)

Ver docs/BASE\_DE\_DATOS.md para el detalle completo de las 22 tablas. Puntos clave:

- exportSchema = false: el esquema versionado a mano vive en database/schema.sql para no depender de un paso de build adicional (JSON de esquema de Room) en el entorno de generación.
- Todas las relaciones usan @ForeignKey(onDelete = CASCADE) e índices explícitos en las columnas FK.
- El contenido semilla (DatabaseSeeder) se ejecuta una única vez al primer arranque (islandDao().count() == 0 como guarda de idempotencia), insertando los 6 objetos Seed\*.kt en orden de dependencia.

### 5. Motores de dominio

#### LogicChallengeEngine

Valida las 3 mecánicas del Laboratorio de Lógica:

- checkSequence: compara el orden propuesto por el usuario contra items.sortedBy { correctPosition }.
- checkMatch: valida que cada par (premisa, conclusión) propuesto comparta pairKey y tenga roles opuestos.
- checkSpotFlaw: compara el id seleccionado contra el único item con isFlawed = true.

#### ReasonCoherenceEngine

Mide cuántas etiquetas de valor (`valueTags`) se repiten entre las cartas de razones seleccionadas y devuelve un nivel (BAJA/MEDIA/ALTA) con un mensaje educativo. Nunca juzga la postura elegida, solo la consistencia interna de las razones.

## ProgressCalculator

Deriva cristales ganados/totales y el `ModuleStatus` (`LOCKED/AVAILABLE/STARTED/COMPLETED/MASTERED`) de una isla a partir de conteos reales de intentos, nunca de un contador incrementado manualmente. El desbloqueo de una isla depende de la suma global de cristales de todas las islas frente a un `unlockRequiredCrystals`.

## BadgeEngine

Compara `ProfileStats` (derivadas por `ProgressRepository` a partir de las tablas de intentos) contra los `unlockCriteriaKey` fijos del catálogo de insignias, devolviendo qué claves se cumplen. `ProgressRepository.recalculateAll()` desbloquea solo las insignias nuevas (no re-inserta las ya desbloqueadas, gracias a `OnConflictStrategy.IGNORE`).

## 6. Flujo de recálculo de progreso

Cualquier repositorio que registra un intento (`DailyQuestionRepository.submitAnswer`, `DilemmaRepository.recordAttempt`, `LogicRepository.recordAttempt`, `PerspectiveRepository.recordAttempt`, `SelfDebateRepository.recordAttempt`, `ReflectionRepository.addJournalEntry/recordOpinionRevision`) llama a `ProgressRepository.recalculateAll()` al final de la operación. Este método:

1. Lee los conteos reales por isla desde cada DAO.
2. Llama a `ProgressCalculator.compute(...)` por isla.
3. Persiste el resultado en `island_progress` (upsert).
4. Evalúa `BadgeEngine` sobre las estadísticas globales y por isla, y desbloquea insignias nuevas.

## 7. Grabación de audio

`util/AudioRecorderManager.kt` usa `android.media.MediaRecorder` (formato `MPEG_4/AAC`) y `android.media.MediaPlayer` reales. Los archivos se guardan en `context.filesDir/reflexiones/` (almacenamiento privado de la app). Duración máxima forzada con `setMaxDuration(45_000)`. No hay transcripción ni subida a ningún servidor.

## 8. Navegación

`ui/navigation/Routes.kt` centraliza las 18 rutas (algunas con argumentos `{islandId}`, `{dilemmaId}`, etc.). `FilosofarNavHost.kt` construye el `NavHost` y decide la pantalla inicial (`Routes.MAP` si el perfil ya completó onboarding, `Routes.ONBOARDING` si no) leyendo `UserProfileRepository.observeProfile()` de forma reactiva.

## 9. Dependencias completas

Ver `app/build.gradle.kts` para la lista exacta con versiones. Resumen por función:

- UI: Compose BOM, Material3, Navigation Compose, ui-tooling.
- Datos: Room runtime/ktx/compiler (KSP), DataStore Preferences (reservado para configuración futura).
- Concurrencia: `kotlinx-coroutines-android`.
- Pruebas: JUnit4, `kotlinx-coroutines-test`, Turbine, Room testing, Robolectric, AndroidX Test Core, Truth, Espresso, Compose UI Test.

## 10. Permisos declarados

Solo `android.permission.RECORD_AUDIO`, con `<uses-feature android:required="false">` para microphone. Sin INTERNET.

## 11. Build y CI

`.github/workflows/android-build.yml` ejecuta, en cada push: `testDebugUnitTest`, `lintDebug`, `assembleDebug`, y publica el APK de debug como artefacto. Ver `docs/BUILD_REPORT.md` para el estado de verificación local (no disponible en el entorno de generación).

## 12. Mantenimiento y ampliación

- **Añadir contenido:** editar los objetos en `data/local/seed/Seed*.kt`; `DatabaseSeeder` los inserta automáticamente en instalaciones nuevas (no reinserta en dispositivos ya instalados sin borrar datos, por diseño de idempotencia).
- **Añadir una isla:** agregar entrada a `SeedIslands.all` y contenido asociado con el mismo `islandId` en los demás `Seed*.kt`.
- **Añadir un tipo de reto de lógica:** extender el enum `LogicChallengeType`, añadir un caso en `LogicChallengeEngine`, y una rama de UI en `LogicChallengeScreen`.
- **Cambiar la paleta visual:** `ui/theme/Color.kt` centraliza todos los colores; los iconos por isla usan `themeColorHex` desde la entidad, no colores hardcodeados en la UI.